> # CST1510 Programming for data communication and networks
>
> ## Week 5: File Handling and Data Structures including Pandas
>
> ## WORKSHOP

I. File Handling

Files

- A **text file** consists of a series of characters.
- A **binary file** (used for images, videos, etc.) consists of a series of bytes.
- The first character in a text file, or byte in a binary file, is at position 0.
  - In a file with ***n*** characters or bytes, the last position is ***n* – 1**.
- When you **open** a file in Python, it creates a **file object** that allows you to interact with the file.

No description has been provided for this image

1. File Handling Functions

- `open()` : Opens a file and returns a file object.
  - Parameters:
    - `filename` : Name of the file.
    - `mode` : The mode in which the file is opened (`'r'` , `'w'` , `'a'` , etc.).

```
file = open('file.txt', 'r')  # Opens file in read mode
```

2. The `with` Statement (Writing to a Text File)

- Python's file handling capabilities are built on the **`io` module**, which provides a range of classes for working with files.
- The `with` statement:
  - Acquires a resource and assigns its corresponding object to a variable.
  - Enables the application to use the resource through that variable.
  - Automatically calls the resource object's **`close` method** to release the resource when done.

In [116…

```
# Example:
with open('accounts.txt', mode='w') as f:
    f.write('101 Smith 150.75\n')
    f.write('202 Johnson 89.34\n')
    f.write('303 Williams 452.00\n')
    f.write('404 Brown -23.45\n')
    f.write('505 Davis 320.89\n')
```

Activity: 1 Writing to a File

Write a Python program that writes your name and today's date into a new file called user_info.txt.

Steps:

1. Open a new file in write mode.
2. Write your name and the current date to the file.
3. Close the file after writing.

In [ ]:

```

```

Built-In Function `open`

- The `open` function is used to access a file.
- The `mode` argument determines how the file is accessed:
  - `'r'` : Reading (raises error if the file does not exist).
  - `'w'` : Writing (overwrites if the file exists).
  - `'a'` : Appending (adds new data to the end of the file).

In [ ]:
```
with open('accounts.txt', 'r') as f:
    data = f.read()
```

File Path

- If the file is in the same directory as your Python script, simply provide the file name:

In [ ]:
```
with open('file.txt', 'r') as f:
    pass
```

- For files in different directories, specify the full path:
  - **Mac**: `"/Users/anna29/PycharmProjects/CST1510/accounts.txt"`
  - **Windows**: `r"C:\Users\anna29\PycharmProjects\CST1510\accounts.txt"`

Activity 2 : Reading a File

Write a Python program that reads and prints the content of `user_info.txt``.

Steps:

1. Open the `user_info.txt` file in read mode.
2. Read and print the content of the file.

In [ ]:

```

```

4. Writing to a File

- `write` : Writes a string to the file.
- `writelines` : Takes an iterable and writes each element to the file.

In [117…

```
with open('accounts.txt', mode='w') as f:
    f.writelines(['101 Smith 150.75\n', '202 Johnson 89.34\n'])
```

File Object Methods

- `write()` : Writes a string to the file.
- `writelines()` : Writes a list of strings to the file.
- `read()` : Reads the entire file content.
- `readline()` : Reads a single line from the file.
- `readlines()` : Reads the entire file into a list of strings.

Activity 3: Writing to a File Using a List

Write a Python program that stores a list of your favorite books in a file called `books.txt` .

Steps:

1. Create a list of book titles.
2. Write each book title into the file using writelines().

In [ ]:

```

```

5. Reading from a File

Reading 10 Characters

In [7]:
```
# Example:
with open('accounts.txt', mode='r') as f:
    print(f.read(10))  # Reads 10 characters from the file
```

```
101 Smith
```

Reading Line-by-Line

In [118…
```
with open('accounts.txt', mode='r') as accounts:
    for record in accounts:
        account, name, balance = record.split()
        print(f'{account:<10}{name:<10}{balance:>10}')
```

```
101       Smith         150.75
202       Johnson        89.34
```

Activity 4: Reading Specific Characters from a File

Write a Python program that reads the first 25 characters from Les_Brown.txt.

Steps:

1. Open Les_Brown.txt in read mode.
2. Use the read() method to read 25 characters.
3. Print the result.

6. File Modes

Mode	Description
`'r'`	Opens a text file for reading. This is the default mode if none is specified when calling `open`.
`'w'`	Opens a text file for writing. Any existing content is **erased**.
`'a'`	Opens a text file for appending. If the file doesn't exist, it is created. New data is added to the end of the file.

Mode	Description
`'r+'`	Opens a text file for both reading and writing.
`'w+'`	Opens a text file for both reading and writing. Any existing content is **erased**.
`'a+'`	Opens a text file for both reading and appending. If the file doesn't exist, it is created. New data is added to the end of the file.

Activity 5: Appending to a File

Write a Python program that opens the user_info.txt file and appends your address to the end.

Steps:

1. Open user_info.txt in append mode.
2. Write your address at the end of the file.
3. Close the file.

7. Handling File Paths

Example: Using File Paths

- **MAC example:**

In [ ]:
```
path = "/Users/anna29/PycharmProjects/CST1510/accounts.txt"
```

- **WINDOWS example:**

In [ ]:
```
path = r"C:\Users\anna29\PycharmProjects\CST1510\accounts.txt"
```

In [ ]:
```
with open(path, "r") as f:
    data = f.readline()
```

Activity 6: Handling File Not Found Errors

Write a Python program that tries to read from a file called missing.txt. If the file is not found, handle the error gracefully by printing a message.

Steps:

1. Use a try and except block to handle FileNotFoundError.
2. Attempt to read from missing.txt.
3. Print an error message if the file does not exist.

In [27]:
```
try:
    with open(path, "r")as f:
        data = f.readline()
except FileNotFoundError as e:
    print(e)
else:
    print(f'{"Account":<10}{"Name":<5}{"Balance":>7}')
    print(data)
```

```
[Errno 2] No such file or directory: 'C:\\Users\\anna29\\PycharmProjects\\CST1510\\accounts.txt'
```

8. File Method: `readlines()`

- The `readlines()` method reads the entire text file and returns a list of strings.
- This works well for small files but is less efficient for large files.

In [ ]:
```
with open('accounts.txt', mode='r') as f:
    lines = f.readlines()
    print(lines)
```

Activity 7: Using readlines()

Write a Python program that reads the content of `Les_Brown.txt` and prints each line separately.

Steps:

1. Open `Les_Brown.txt` using readlines().
2. Print each line from the file.

II. Data Structures

Data structures are essential for designing efficient software and implementing various algorithms. Python provides several built-in data structures that are simple to use and powerful in functionality.

1. Lists

Lists are mutable sequences that can store elements of different data types. They are versatile and provide various methods for manipulation.

Common List Operations

In [43]:
```
# Creating a list
numbers = [1, 3, 4, 5]
```

In [44]:
```
# Adding an element
numbers.append(7)
print("After append:", numbers)  # Output: [1, 3, 4, 5, 7]
```

```
After append: [1, 3, 4, 5, 7]
```

In [45]:
```
# Accessing an element
print("First element:", numbers[0])  # Output: 1
```

```
First element: 1
```

In [46]:
```
# Deleting an element
numbers.pop()  # Removes the last element
```

```
print("After pop:", numbers)  # Output: [1, 3, 4, 5]
```

```
After pop: [1, 3, 4, 5]
```

In [47]:
```
# Reversing the list
numbers.reverse()
print("Reversed list:", numbers)  # Output: [5, 4, 3, 1]
```

```
Reversed list: [5, 4, 3, 1]
```

In [48]:
```
# Summing elements in the list
print("Sum of elements:", sum(numbers))  # Output: 13
```

```
Sum of elements: 13
```

In [49]:
```
# Clearing the entire list
numbers.clear()
print("After clear:", numbers)  # Output: []
```

```
After clear: []
```

Activity 1 List Operations

Write a Python program that:

1. Creates a list of integers from 1 to 10.
2. Adds a new number (15) to the list.
3. Removes the first and last element.
4. Reverses the list and prints the result.

In [ ]:
```

```

2. `Tuples`

- Tuples are immutable sequences, meaning their **contents cannot be changed once created**.
- They help store fixed collections of items.

Creating and Using Tuples

In [51]:
```
# Creating a tuple
x = (1, 2, 3)
```

In [52]:
```
# Creating a tuple with one element
x = 2,
```

In [53]:
```
# Creating an empty tuple
x = ()
```

In [54]:
```
# Casting into a tuple
list1 = [1, 2, 3]
tuple(list1)
```

Out[54]:
```
(1, 2, 3)
```

In [55]:
```
# Creating a tuple list into tuple comprehension
x = tuple ([2 * x for x in range(1,5)])
```

In [56]:
```
# Accessing elements
print("Element at index 0:", x[0])  # Output: 2
```

```
Element at index 0: 2
```

In [57]:
```
# Converting a list to a tuple
list1 = [1, 2, 3]
tuple_from_list = tuple(list1)
print("Tuple from list:", tuple_from_list)  # Output: (1, 2, 3)
```

```
Tuple from list: (1, 2, 3)
```

In [59]:
```
# Tuples are immutable
try:
    x[0] = 10  # This will raise a TypeError
except TypeError as e:
    print("Error:", e)
```

```
Error: 'tuple' object does not support item assignment
```

Add Elements

Standard adding a new element is imposable on Tuples however you can concatenate tuple with another tuple to create a new tuple.

In [60]:
```
x = (1,2,3)
x += (4,) # or x = x + (4,)
x
```

Out[60]:
```
(1, 2, 3, 4)
```

'tuple' object doesn't support item deletion

Tuple is Immutable however member object may be mutable

In [62]:
```
x = (1, 2, 3)
del (x[1]) # TypeError: 'tuple' object doesn't support item deletion
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[62], line 2
      1 x = (1, 2, 3)
----> 2 del (x[1]) # TypeError: 'tuple' object doesn't support item deletion

TypeError: 'tuple' object doesn't support item deletion
```

In [63]:
```
x[1] = 8 # error! Tuples doesn't support item assignment?
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[63], line 1
----> 1 x[1] = 8 # error! Tuples doesn't support item assignment?

TypeError: 'tuple' object does not support item assignment
```

In [64]:
```
x = ([1, 2], 3) # 2-item tuple: list and int?
# 0 1
```

```
temp = x[0]
temp[1]
```

Out[64]: 2

In [65]:
```
x = ([1, 2], 3)
# 0 1
# 0 1
del (x[0][1])
print(x)
```

```
([1], 3)
```

Activity 2 Tuple Operations

Write a Python program that:

1. Creates a tuple containing the names of five different cities.
2. Prints the third city.
3. Concatenates a new tuple containing two more cities.
4. Converts the tuple to a list and prints the result.

In [ ]:
```

```

3. `Sets`

- Sets are a collection of **distinct (unique) objects**.
- Sets behave as collections of dictionary keys without any associated values
- These are useful to create lists that only hold unique values in the dataset.
- It is an unordered but mutable collection; this is very helpful when going through a vast dataset.

Common Set Operations

In [67]:
```
# Creating a set
x = {3, 5, 3, 3}
print("Set with unique elements:", x)  # Output: {3, 5}
```

```
Set with unique elements: {3, 5}
```

In [68]:
```
# Converting a list to a set to remove duplicates
list1 = [1, 2, 1, 4]
x = set(list1)
print("Set from list:", x)  # Output: {1, 2, 4}
```

```
Set from list: {1, 2, 4}
```

In [69]:
```
# Basic set operations
x = {18, 21, 24, 27}
```

In [70]:
```
# Adding an item
x.add(30)
print("After add:", x)  # Output: {18, 21, 24, 27, 30}
```

```
After add: {18, 21, 24, 27, 30}
```

In [71]:
```
# Removing an item
x.remove(18)
print("After remove:", x)  # Output: {21, 24, 27, 30}
```

```
After remove: {21, 24, 27, 30}
```

In [72]:
```
# Check membership
print("21 in set:", 21 in x)  # Output: True
```

```
21 in set: True
```

In [ ]:
```
# Remove last element
x.pop()
print(x)
```

List of functions on sets:

In [73]:
```
set1 = {1, 2, 3, 4, 5}
set2 = {4, 5, 6, 7, 8}
```

Intersection (`AND`)

In [74]:
```
print(set1 & set2 )
```

```
{4, 5}
```

In [75]:
```
print(set1.intersection(set2))
```

```
{4, 5}
```

Union (`OR`)

In [76]:
```
print(set1 | set2)
```

```
{1, 2, 3, 4, 5, 6, 7, 8}
```

In [77]:
```
print(set1.union(set2))
```

```
{1, 2, 3, 4, 5, 6, 7, 8}
```

Symmetric Difference (`XOR`)

In [78]:
```
print( set1 ^ set2)
```

```
{1, 2, 3, 6, 7, 8}
```

In [79]:
```
print( set1.symmetric_difference(set2))
```

```
{1, 2, 3, 6, 7, 8}
```

Difference (In set1 but not in set2)

In [80]:
```
print(set1 - set2)
```

```
{1, 2, 3}
```

In [81]:
```
print(set1.difference(set2))
```

```
{1, 2, 3}
```

In [82]:
```
print(set1)
set1.discard(3)
print(set1)
```

```
{1, 2, 3, 4, 5}
{1, 2, 4, 5}
```

Frozen set

The frozenset() function returns an unchangeable frozenset object (which is like a set object, only unchangeable)

In [83]:
```
mylist = [18, 21, 24, 27, 30]
x= frozenset(mylist)
print(x)
```

```
frozenset({18, 21, 24, 27, 30})
```

Activity 3: Set Operations

Write a Python program that:

1. Creates a set of numbers from 1 to 10.
2. Adds two new numbers to the set.
3. Removes a number from the set.
4. Finds the intersection and union between this set and another set of numbers from 5 to 15.

In [ ]:
```

```

4. `Dictionaries`

Dictionaries store data as key-value pairs, making them ideal for associating related information.

Common Dictionary Operations

In [84]:
```
# Creating a dictionary with a single key-value pair
x = {'tomatoes': 15.5, 'onions': 3.50, 'mushrooms': 1.50}
```

In [85]:
```
# Creating a dictionary using the `dict()` function with a list of tuples
x = dict([('tomatoes', 15.5), ('onions', 3.50), ('mushrooms', 1.50)])
```

In [86]:
```
# Creating a dictionary using the `dict()` function with keyword arguments
x = dict(tomatos= 15.5, onions= 3.50, mushrooms= 1.50)
```

In [87]:
```
# Adding or updating an item
x['tomatoes'] = 25.2
print("Updated dictionary:", x)  # Output: {'tomatoes': 25.2, 'onions': 3.5, 'mushrooms': 1.5}
```

```
Updated dictionary: {'tomatos': 15.5, 'onions': 3.5, 'mushrooms': 1.5, 'tomatoes': 25.2}
```

In [88]:
```
# Removing an item
del x['onions']
print("After delete:", x)  # Output: {'tomatoes': 25.2, 'mushrooms': 1.5}
```

```
After delete: {'tomatos': 15.5, 'mushrooms': 1.5, 'tomatoes': 25.2}
```

In [89]:
```
# Remove the last item from dict
x.popitem()
print(x)
```

```
{'tomatos': 15.5, 'mushrooms': 1.5}
```

In [90]:
```
# Check membership in x (only looks in keys, not values)
'mushrooms' in x
```

Out[90]:
```
True
```

In [91]:
```
'mushrooms' not in x
```

Out[91]: False

Accessing elements in a dictionary

In [92]:
```
x = {'tomatoes': 15.5, 'onions': 3.50, 'mushrooms': 1.50}
```

In [93]:
```
# returns list of keys in x?
x.keys()
```

Out[93]: dict_keys(['tomatoes', 'onions', 'mushrooms'])

In [94]:
```
# returns list of values in x?
x.values()
```

Out[94]: dict_values([15.5, 3.5, 1.5])

In [95]:
```
# returns list of key-value tuple pairs in x?
x.items()
```

Out[95]: dict_items([('tomatoes', 15.5), ('onions', 3.5), ('mushrooms', 1.5)])

In [98]:
```
# iterate key/values pair
for key,value in x.items():
    print(key, value)
```

```
tomatoes 15.5
onions 3.5
mushrooms 1.5
```

In [99]:
```
# Iterating over key-value pairs
for key, value in x.items():
    print(f"{key}: {value}")
    # Output:
    # tomatoes: 25.2
    # mushrooms: 1.5
```

```
tomatoes: 15.5
onions: 3.5
mushrooms: 1.5
```

In [100…

```python
# iterate over keys
for k in x:
    print(k)
```

```
tomatoes
onions
mushrooms
```

In [101…

```python
# iterate values
for v in x.values():
    print(v)
```

```
15.5
3.5
1.5
```

Activity 4: Dictionary Operations

Write a Python program that:

1. Creates a dictionary of 5 fruits and their prices.
2. Updates the price of one fruit.
3. Deletes a fruit from the dictionary.
4. Iterates over the dictionary and prints each fruit and its price.

In [ ]:

```python

```

5. Pandas

Pandas provides data structures like Series and DataFrame for efficiently handling and analyzing data. It's a powerful and easy-to-use data analysis and manipulation library for Python.

Installing Pandas

To get started, you need to install Pandas if you haven't already:

copy and run the line below to install Pandas if not installed

```
# !pip install pandas # for windoes users
# !pip3 install pandas # for mac users
```

Importing Pandas

In [103…

```
import pandas as pd  # Import Pandas with the alias 'pd'
```

Creating Data Structures

Series

A Series is a one-dimensional array-like object that can hold data of any type

In [111…

```
# Creating a Series
data = [10, 20, 30, 40]
index=['a', 'b', 'c', 'd']

series = pd.Series(data,index)
print(series)
```

```
a    10
b    20
c    30
d    40
dtype: int64
```

DataFrame

A DataFrame is a two-dimensional table-like data structure with labeled axes (rows and columns).

In [112…

```python
# Creating a DataFrame from a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}


df = pd.DataFrame(data)
print(df)
```

```
     Name  Age         City
0   Alice   25     New York
1     Bob   30  Los Angeles
2 Charlie   35      Chicago
```

Reading Data from Files

Pandas make it easy to read data from different file formats such as CSV, Excel, and JSON.

Reading a CSV File

In [ ]:

```python
# Read data from a CSV file
# df = pd.read_csv('data.csv')  # Uncomment and replace 'data.csv' with your file path
```

Reading an Excel File

In [ ]:

```python
# Read data from an Excel file
# df = pd.read_excel('data.xlsx')  # Uncomment and replace 'data.xlsx' with your file path
```

Basic DataFrame Operations

Viewing Data

In [ ]:
```
# Display the first few rows
print(df.head())  # Shows the first 5 rows by default
```

In [ ]:
```
# Display the last few rows
print(df.tail())  # Shows the last 5 rows by default
```

Descriptive Statistics

In [ ]:
```
# Get a summary of numerical columns
print(df.describe())
```

In [ ]:
```
# Get a summary of numerical columns
print(df.describe())
```

Selecting Data

In [ ]:
```
# Select a column
print(df['Name'])
```

In [ ]:
```
# Select multiple columns
print(df[['Name', 'City']])
```

In [ ]:
```
# Select rows by index
print(df.iloc[0:2])  # Selects the first two rows
```

Filtering Data

In [ ]:
```
# Filter rows based on a condition
adults = df[df['Age'] > 30]
print(adults)
```

Modifying Data

Adding a New Column

In [ ]:
```
# Add a new column 'Salary' to the DataFrame
df['Salary'] = [50000, 60000, 70000]
print(df)
```

Updating Values

In [ ]:
```
# Update the 'City' of Bob to 'San Francisco'
df.loc[df['Name'] == 'Bob', 'City'] = 'San Francisco'
print(df)
```

Removing Columns and Rows

In [ ]:
```
# Remove a column
df = df.drop('Salary', axis=1)  # axis=1 indicates columns
print(df)
```

In [ ]:
```
# Remove a row by index
df = df.drop(1, axis=0)  # axis=0 indicates rows
print(df)
```

Activity 5 (Threshold): Pandas DataFrame

Write a Python program that:

1. Creates a Pandas DataFrame with columns for "Name", "Age", and "Country".
2. Adds a new column "Salary" to the DataFrame.
3. Selects and prints rows where the age is greater than 30.

In [ ]:
```

```

Handling Missing Data

Detecting Missing Data

In [ ]:
```
# Check for missing values
print(df.isnull().sum())
```

Filling Missing Values

In [ ]:
```
# Fill missing values with a specific value
df.fillna('Unknown', inplace=True)
```

Dropping Missing Values

In [ ]:
```
# Drop rows with missing values
df.dropna(inplace=True)
```

Grouping and Aggregation

Grouping data allows you to group rows based on column values and perform aggregate functions.

In [ ]:
```
# Group by 'City' and calculate mean age
grouped = df.groupby('City')['Age'].mean()
print(grouped)
```

Saving Data to a File

Saving to CSV

In [ ]:
```
# Save the DataFrame to a CSV file
df.to_csv('output.csv', index=False)
```

Saving to Excel

In [ ]:
```
# Save the DataFrame to an Excel file
df.to_excel('output.xlsx', index=False)
```

In [ ]:
```
```